

Content

Content

1. About Frank Castle 🦀
2. Disclaimer
3. Risk Classification
4. Audit Scope
5. Summary of Findings
6. Findings
 1. Critical findings
 2. High findings
 3. Medium Findings
 4. Low findings
 5. Informational Findings

About Frank Castle 🦀

Frank Castle is a professional smart contract security researcher with a focused expertise in auditing Rust-based contracts and decentralized infrastructure across leading blockchain ecosystems, including Solana , Polkadot , and Cosmos (CosmWasm). 🦀

Frank Castle has audited Lido, GMX ,Pump.fun, LayerZero, Synthetix , Hydration, something.cool ,DUB Social and several multi-million protocols.

with more than ~35 Rust Audit , ~25 Solana Audits , and +200 criticals/highs found , All the reports can be found [here](#)

For private audit or consulting requests please reach out to me via Telegram @[castle_chain](#) , **Twitter** ([@0xfrank_auditor](#)) or **Discord** (castle_chain).

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

Summary of Findings

ID	Title	Severity	Status
[M-01]	Re-Accepting Dare Bypasses Migration Lock and Reopens Trading	Medium	Resolved
[M-02]	Mint Account Space Calculation Does Not Account for Token Extensions	Medium	Resolved
[M-03]	Rounded-Down Fee Calculation Allows Fee-Free Trades	Medium	Resolved
[M-04]	Slippage Check Does Not Account for Trading Fees	Medium	Acknowledged
[L-01]	Initialization Frontrunning Vulnerability Due to Permissionless `Initialize` Instruction	Low	Acknowledged
[L-02]	Missing Lamports Check Allows DoS via Pre-Funded Account	Low	Resolved

Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Audit Scope

Dare Program : <https://github.com/se1250/dareprogram>

Findings

1. Medium Findings

[M-01] Re-Accepting Dare Bypasses Migration Lock and Reopens Trading

Description

The current implementation allows the `DareTradeStatus` to be reset to `Open` during a dare re-acceptance flow, **even when trading was previously stopped due to critical constraints**, such as hitting the `MigrationRequired` state. This logic flaw enables users to bypass intended restrictions and reopen trading inappropriately.

The issue arises from the following code:

```
if submitters_last_idx == 0 {  
    dare_state.dare_status = DareStatus::Open;  
    dare_state.open_timestamp = clock.unix_timestamp as u64;  
}
```

This resets the dare to an open state if all previous submitters were cleared (e.g., via `complete_dare`). However, **no check is made to preserve or validate the previous `DareTradeStatus`**, which may have been `MigrationRequired` or even `Banned`.

Subsequently, when a user calls `accept_dare`, the code sets:

```
dare_state.dare_status = DareStatus::Accepted;  
dare_state.trade_status = DareTradeStatus::Open;
```

This allows trading to be reopened **even if it was previously closed or restricted due to migration conditions**.

Proof of Concept

Initial State (State 0):

- `DareTradeStatus` = `MigrationRequired` (migration threshold reached)
- `DareStatus` = `Accepted`

Action 1: Moderator calls `complete_dare` multiple times

- Each call marks a submitter as unsuccessful, removing them from the `submitters` list.
- After the last submitter is removed:
 - `DareStatus` is set to `Open`
 - `open_timestamp` is updated

Reference:

[\[Link to complete.rs code\]](#)

https://github.com/se1250/dareprogram/blob/fd6780d309e250a3d82e140c57a3c86542220431/programs/dare_tokens/src/instructions/complete.rs#L158-L161

State After Action 1 (State 1):

- `DareTradeStatus` = `MigrationRequired`
- `DareStatus` = `Open`

Action 2: Any user calls `accept_dare`

- This sets:
 - `DareStatus` = `Accepted`
 - `DareTradeStatus` = `Open` ✓

Final State (State 2):

- `DareTradeStatus` = `Open` ✓
- `DareStatus` = `Accepted` ✓

Impact:

With trading reopened:

- Tokens can be **bought and sold again**
- The **reserve may drop below the migration threshold**
- The **migration process can be stalled or bypassed entirely**
- Banned or migrated dares can be **manipulated back into circulation**

This effectively **nullifies the protections imposed by** `MigrationRequired` or other critical trade states, re-enabling trading when it should be permanently or temporarily locked.

Recommendations

Update the logic to ensure that reopening the dare **does not automatically reopen trading**. Specifically:

- Only allow the `trade_status` to transition to `Open` if the previous value was explicitly `Closed`, and never if it was `MigrationRequired` or `Banned`.

Example fix:

```
if dare_state.trade_status == DareTradeStatus::Closed {  
    dare_state.trade_status = DareTradeStatus::Open;  
}
```

[M-02] Mint Account Space Calculation Does Not Account for Token Extensions

Description

When creating the mint account, the code fails to account for the space required by token extensions — specifically, the `MetadataPointer` extension. The `token_extensions` vector is initialized as empty, so the calculated mint space does not reserve sufficient space for extensions. This results in incorrect mint space allocation and will cause the token mint creation to fail at runtime.

Example from the code:

```
let mut token_extensions: Vec<ExtensionType> = vec![];  
  
let mint_space =  
    ExtensionType  
    ::try_calculate_account_len  
    ::<spl_token_2022::state::Mint>(  
        &token_extensions  
    )?;  
let mint_balance = ctx.accounts.rent.minimum_balance(mint_space).max(1);  
  
system_program::create_account(CpiContext::new(  
    ctx.accounts.system_program.to_account_info(),  
    system_program::CreateAccount {  
        from: ctx.accounts.user.to_account_info(),  
        to: ctx.accounts.token_mint.to_account_info()  
    }  
)
```

```

    ),
    mint_balance,
    mint_space as u64,
    &ctx.accounts.config.token_program
)?;

```

Recommendations

Update the logic to include necessary extensions, such as `MetadataPointer`, when calculating the mint space. For example:

```

if let Some(metadata_authority) = ctx.accounts.config.metadata_authority {
    token_extensions.push(ExtensionType::MetadataPointer);
}

let mint_space =
    ExtensionType
    ::try_calculate_account_len
    ::<spl_token_2022::state::Mint>(
        &token_extensions
    );
let mint_balance = ctx.accounts.rent.minimum_balance(mint_space).max(1);

system_program::create_account(CpiContext::new(
    ctx.accounts.system_program.to_account_info(),
    system_program::CreateAccount {
        from: ctx.accounts.user.to_account_info(),
        to: ctx.accounts.token_mint.to_account_info()
    }
),
    mint_balance,
    mint_space as u64,
    &ctx.accounts.config.token_program
)?;

```

This ensures that the mint account is allocated with sufficient space for the intended extensions, avoiding runtime failures during mint creation.

[M-03] Rounded-Down Fee Calculation Allows Fee-Free Trades

Description

In the trading functions `buy` and `sell`, the fee is calculated as:

```
effective_sol_spend * fees_bps / BPS
```

Because integer division is used, this calculation is rounded down. In cases where the `effective_sol_spend` value is small, the resulting fee can be zero, allowing users to perform fee-free trades by splitting large trades into multiple small ones.

Example scenario:

- `fee_bps = 100`
- `effective_sol_spend = 99`

The fee calculation would be:

```
(99 * 100) / 10_000 = 9900 / 10_000 = 0
```

Thus, no fee is charged. This creates an incentive for users to split large trades into smaller ones to avoid paying fees.

Relevant code snippet:

```
let trading_fee =
  effective_sol_spend
    .checked_mul(ctx.accounts.config.trade_fee_bps)
    .ok_or(DareErrors::MathOverflow)? / BPS;
```

Recommendations

Consider implementing a minimum fee of at least 1 unit when the `effective_sol_spend` is non-zero. or apply rounding up instead of down to ensure all trades contribute to fees as intended.

[M-04] Slippage Check Does Not Account for Trading Fees

Description

In the trading functions, the slippage check is performed against the `effective_sol_spend` amount only, without considering the additional trading fee. As a result, users could end up paying more SOL than their specified slippage tolerance allows, because the slippage validation does not account for the total SOL deducted (i.e., `effective_sol_spend + trading_fee`).

Current code:

```
let trading_fee =
  effective_sol_spend
    .checked_mul(ctx.accounts.config.trade_fee_bps)
    .ok_or(DareErrors::MathOverflow)? / BPS;

require!(effective_sol_spend <= pay_cap, DareErrors::SlippageLimit);
```

This validation only checks the `effective_sol_spend`, but users actually pay `effective_sol_spend + trading_fee`.

Recommendations

Update the slippage validation to include the trading fee in the comparison:

```
require!(effective_sol_spend + trading_fee <= pay_cap,  
DareErrors::SlippageLimit);
```

This ensures that users never pay more SOL (including fees) than their slippage limit permits, aligning the validation with the actual amount charged in the trade.

2. Low Findings

[L-01] Initialization Frontrunning Vulnerability Due to Permissionless `initialize` Instruction

Description

The `initialize` instruction in the Dare program is currently **permissionless**, allowing **any user** to invoke it immediately after deployment. This permits the first caller to set the `owner` field of the configuration account, potentially locking out the intended owner or misconfiguring critical parameters.

This behavior exposes the program to **frontrunning attacks**, where a malicious actor can take control of the initialization process by front-running the intended deployer.

```
pub struct InitAccounts<'info> {  
    #[account(  
        init,  
        payer = owner,  
        seeds = [CONFIG_PDA_SEED],  
        bump,  
        space = 8 + size_of::<Configuration>(),  
        rent_exempt = enforce  
    )]  
    pub config: Box<Account<'info, Configuration>>,  
  
    // TODO: ensure that owner is our wallet  
    #[account(mut)]  
    pub owner: Signer<'info>,  
  
    pub system_program: Program<'info, System>,  
}
```


Recommendation

It is recommended to implement **access control** in the `initialize` instruction to restrict who can perform initialization. One secure approach is to require the instruction to be co-signed by the program's upgrade authority (a keypair known only to the deployer), ensuring only the legitimate owner can initialize the config:

```
// Require 2 signatures for initialization
#[account(mut, signer)]
pub authority: Signer<'info>,

// Ensure the signer matches the program's ID
#[account(address = crate::ID)]
pub program: Signer<'info>,
```

[L-02] Missing Lamports Check Allows DoS via Pre-Funded Account

Description

The code does not check whether the target account already holds lamports before attempting to create the mint. This check is critical because, in such cases, the logic should branch into manually creating the account using `transfer`, `allocate`, and `assign` calls.

The absence of this manual creation step can lead to a denial-of-service (DoS) scenario during mint creation. Anyone can pre-fund a PDA by sending it lamports, making the account ineligible for creation using `create_account`. As a result, the creation attempt will be rejected due to the following logic in the system program:

https://github.com/anza-xyz/agave/blob/a073364d22a4cb12006cc9ff68bd41a60feec91/programs/system/src/system_processor.rs#L162-L169

Recommendations

If the account already holds some lamports, manually create the account using `transfer`, `allocate`, and `assign` calls, as shown below. Alternatively, consider using the `init_if_needed` Anchor constraint to handle pre-funded accounts safely.

```
// If the account has some lamports already it can't be created using
// create_account instruction.
// Anybody can send lamports to a PDA and by doing so create the account
// and perform DoS attack by blocking create_account
if account_info.lamports() > 0 {
```

```

    let top_up_lamports =
total_lamports.saturating_sub(account_info.lamports());

    if top_up_lamports > 0 {
        invoke(
            &system_instruction::transfer(payer_info.key, account_info.key,
top_up_lamports),
            &[
                payer_info.clone(),
                account_info.clone(),
                system_info.clone(),
            ],
        )?;
    }

    invoke_signed(
        &system_instruction::allocate(account_info.key, account_size as
u64),
        &[account_info.clone(), system_info.clone()],
        &[&signers_seeds[..]],
    )?;

    invoke_signed(
        &system_instruction::assign(account_info.key, owner_program_id),
        &[account_info.clone(), system_info.clone()],
        &[&signers_seeds[..]],
    )?;
} else {
    // If the PDA doesn't exist use create_account to use lower compute
budget
    let create_account_instruction = create_account(
        payer_info.key,
        account_info.key,
        total_lamports,
        account_size as u64,
        owner_program_id,
    );

    invoke_signed(
        &create_account_instruction,
        &[
            payer_info.clone(),
            account_info.clone(),

```

```
        system_info.clone(),  
        ],  
        &[&signers_seeds[..]],  
    )?;  
}
```